

开发机外网访问与 Codex 代理配置完整指南

开发机外网访问与 Codex 代理配置完整指南

核心问题澄清

代理不是因为 A100 显卡需要，而是因为开发机上的程序需要访问外网 / API/GitHub/npm/pip，但开发机本身可能没有可用外网。

A100 只是显卡。真正的网络结构问题是：

代码块

- 1 你能ssh dev进去
- 2 ≠
- 3 dev机器能访问GitHub/Codex API/blackaicoding.com/npm/pip

典型的开发机网络结构：

代码块

- 1 你的电脑 —ssh—> 学校/平台开发机
- 2 开发机 —X—> 外网 GitHub / API / npm / OpenAI/Anthropic

群里提到的 "端口转发" 代理 "FlClash" mihomo/geosite.dat 等操作，本质都是解决 \| 开发机借用本机代理访问外网 \| 这个问题。

1. 为什么需要代理？

在开发机上运行 Codex 的网络链路：

代码块

- 1 开发机上的codex
- 2 → blackaicoding.com/v1
- 3 → 第三方API/中转站
- 4 → 模型服务

如果开发机不能直接访问 blackaicoding.com/v1，Codex 会报以下错误：

```
代码块
1  timeout
2  connection refused
3  could not resolve host
4  401 / 403 / 502 / 504 等
```

解决方案：让开发机 "借用你本机的 Clash Verge Dev"

最终目标网络链路：

```
代码块
1  开发机Codex
2  → 开发机127.0.0.1:17890
3  → SSH反向隧道
4  → 你本机127.0.0.1:7890
5  → Clash Verge Dev
6  → blackaicoding.com/v1
```

这就是群里说的 "转发端口"。

2. 为什么是 "反向转发"？

因为代理在**你本机**，但需要用代理的是**远程开发机**。

错误做法：

```
代码块
1  # 开发机上访问127.0.0.1:7890
2  # 这会访问开发机自己的7890端口，而不是你电脑的7890端口
```

正确做法：在你本机执行 SSH 反向端口转发命令

```
代码块
1  ssh -N -R 17890:127.0.0.1:7890 dev
```

命令含义：

```
代码块
1  在远程dev机器上开启一个127.0.0.1:17890端口
```

- 2 远程访问这个端口时，流量会通过SSH隧道转发到你本机的127.0.0.1:7890端口

开发机上的代理环境变量设置：

代码块

```
1 export HTTP_PROXY=http://127.0.0.1:17890
2 export HTTPS_PROXY=http://127.0.0.1:17890
3 export ALL_PROXY=http://127.0.0.1:17890
```

3. 最稳操作流程

第一步：本机确认 Clash Verge Dev 端口

Windows PowerShell 测试命令：

代码块

```
1 curl.exe -I -x http://127.0.0.1:7890 https://api.github.com
2 curl.exe -I -x http://127.0.0.1:7890 https://blackaicoding.com/v1/models
```

如果 7890 端口不通：

1. 打开 Clash Verge Dev 设置
2. 查看 Settings → Mixed Port

常见 Clash 端口：

- 7890
- 7897
- 7899
- 8080

多端口测试命令：

代码块

```
1 curl.exe -I -x http://127.0.0.1:7890 https://api.github.com
2 curl.exe -I -x http://127.0.0.1:7897 https://api.github.com
3 curl.exe -I -x http://127.0.0.1:7899 https://api.github.com
```

假设最终确定本机 Clash 端口是 7890。

第二步：本机开启 SSH 反向隧道

在你本机终端执行：

代码块

```
1  ssh -N -R 17890:127.0.0.1:7890 dev
```

重要提示：

- 这个终端窗口不要关闭
- 没有任何输出是正常现象

永久配置（推荐）：

将以下内容添加到本机 `~/.ssh/config` 文件：

代码块

```
1  Host dev
2      HostName <原来dev对应的地址>
3      User <你的用户名>
4      Port 22
5      IdentityFile ~/.ssh/<你的私钥>
6      IdentitiesOnly yes
7      RemoteForward 17890 127.0.0.1:7890
8      ExitOnForwardFailure yes
9      ServerAliveInterval 30
10     ServerAliveCountMax 3
```

之后只需执行 `ssh dev`，隧道就会自动建立。

VS Code Remote SSH 用户注意：

必须将上述配置写入 `~/.ssh/config`，否则 VS Code 远程终端可能不会携带端口转发。

第三步：在开发机测试代理

另开一个终端，正常进入开发机：

代码块

```
1  ssh dev
```

在开发机上执行：

代码块

```
1 export HTTP_PROXY=http://127.0.0.1:17890
2 export HTTPS_PROXY=http://127.0.0.1:17890
3 export ALL_PROXY=http://127.0.0.1:17890
4
5 curl -I https://api.github.com
6 curl -I https://blackaicoding.com/v1/models
```

结果判断：

返回结果	含义
HTTP/2 200	网络完全通畅
HTTP/1.1 401	网络通畅，只是请求没有携带 API 密钥
HTTP/1.1 403	网络通畅，但服务端拒绝访问 / 风控拦截
curl: (7) Failed to connect to 127.0.0.1 port 17890	SSH 隧道没有建立成功
curl: (28) Operation timed out	代理或外网不通
Could not resolve host	DNS 解析失败或代理未生效

只要不是连接失败或超时，就说明方向正确。

4. 开发机要不要装 FI Clash?

不优先安装。

推荐方案：

代码块

```
1 本机Clash Verge Dev
2  +
3  ssh -R反向转发
4  +
```

推荐理由：

1. 开发机可能没有 GUI，FI Clash 不好用
2. 开发机可能本身没有外网，下载 FI Clash 就很困难
3. 不应该将你的订阅链接放到公共 / 临时开发机上
4. 本机 Clash 已经配置好，直接转发更安全

只有在以下情况才考虑在开发机安装 FI Clash/mihomo：

1. SSH 反向转发被服务器禁止
2. 开发机是你长期独占使用
3. 你能安全地放置订阅链接
4. 开发机有权限运行代理客户端

5. 如果反向转发被禁止怎么办？

如果执行反向转发命令时出现：

代码块

```
1 remote port forwarding failed
2 administratively prohibited
```

说明开发机 SSH 服务禁止了远程端口转发。

备选方案：

1. **方案 A：**询问管理员 / 老师是否可以开启 `AllowTcpForwarding`
2. **方案 B：**在开发机直接配置自己的代理 / mihomo
3. **方案 C（最稳）：**只在本机运行 Codex，让本机 Codex 通过 `ssh dev` 操作远程
4. **方案 D：**本地 clone / 修改代码，然后通过 `rsync` 同步到开发机运行

方案 C 详细说明：

代码块

- 1 Codex 运行在你本机（有网络）
- 2 本机 Codex 通过 `ssh dev` 执行远程命令
- 3 远程机器只负责训练 / 跑代码

这种方案下，开发机甚至不需要安装 Codex。

6. 传输 Codex 到开发机的注意事项

不能只传输 `~/.codex/config.toml` 和 `auth.json`。必须同时满足三个条件：

1. 开发机有 `codex` 可执行文件
2. 开发机有 `~/.codex/config.toml` 和 `auth.json`
3. 开发机能通过直连或代理访问 blackaiaicoding.com/v1

只有前两个条件，没有第三个，Codex 仍然无法使用。

7. 完整命令执行顺序

本机操作：

1. 打开 Clash Verge Dev，确认端口是 7890
2. 本机终端执行：

代码块

```
1  ssh -N -R 17890:127.0.0.1:7890 dev
```

开发机操作：

1. 另开终端进入开发机：

代码块

```
1  ssh dev
```

2. 设置代理环境变量：

代码块

```
1  export HTTP_PROXY=http://127.0.0.1:17890
2  export HTTPS_PROXY=http://127.0.0.1:17890
3  export ALL_PROXY=http://127.0.0.1:17890
```

3. 测试网络连通性：
-

```
1 curl -I https://api.github.com
2 curl -I https://blackaicoding.com/v1/models
```

4. 如果网络通畅，安装 Codex：

代码块

```
1 mkdir -p ~/.local/bin
2 export PATH="$HOME/.local/bin:$PATH"
3
4 ARCH="$(uname -m)"
5 if [ "$ARCH" = "x86_64" ]; then
6     ASSET="codex-x86_64-unknown-linux-musl.tar.gz"
7 elif [ "$ARCH" = "aarch64" ] || [ "$ARCH" = "arm64" ]; then
8     ASSET="codex-aarch64-unknown-linux-musl.tar.gz"
9 else
10    echo "Unsupported architecture: $ARCH"
11    exit 1
12 fi
13
14 TMPDIR="$(mktemp -d)"
15 cd "$TMPDIR"
16
17 curl -fL -o codex.tar.gz
18    "https://github.com/openai/codex/releases/latest/download/$ASSET"
19 tar -xzf codex.tar.gz
20
21 BIN_PATH="$(find "$TMPDIR" -maxdepth 1 -type f -name 'codex-*' | head -n 1)"
22 mv "$BIN_PATH" "$HOME/.local/bin/codex"
23 chmod +x "$HOME/.local/bin/codex"
24
25 echo 'export PATH="$HOME/.local/bin:$PATH"' >> ~/.bashrc
26 codex --version
```

5. 传输本机 Codex 配置到开发机（在本机执行）：

代码块

```
1 scp ~/.codex/config.toml dev:~/.codex/config.toml
2 scp ~/.codex/auth.json dev:~/.codex/auth.json
3 ssh dev 'chmod 600 ~/.codex/config.toml ~/.codex/auth.json'
```

6. 测试 Codex 是否正常工作：


```
1  codex --version
2
3  codex exec \
4    --skip-git-repo-check \
5    --sandbox workspace-write \
6    --ask-for-approval never \
7    "只回复OK，不要修改文件。"
```

8. 最终正确理解

可以概括为：

代码块

- 1 开发机可以ssh进去，说明入站SSH通。
- 2 开发机不一定能访问外网，说明出站网络可能被限制。
- 3 Codex运行在开发机时，需要访问外部API。
- 4 所以需要代理。
- 5 如果代理在本机，就需要ssh -R反向端口转发。
- 6 群里说的一堆操作，本质就是解决"开发机借用本机代理访问外网"。

结论：你说的 "开发机应该没网，所以才需要反向转发" 这个判断大概率是正确的。

需要我把这个文档转换成**可直接复制的 Markdown 文件**，或者帮你生成一个**一键执行的 Shell 脚本**吗？

（注：文档部分内容可能由 AI 生成）